



Name: Muhammad Bilal

Roll No. 22F-3309

Course: Deep Learning

DL Assignment-04

Part A – Theory Questions:

Q1. What is sequential data? Explain its key characteristics and give suitable real-world examples.

Sequential data refers to data points that are ordered and dependent on previous values. In this type of data, the order of the elements matters because the subsequent value may depend on the prior ones.

Key Characteristics:

- **Order dependency:** The current data point depends on previous data points.
- **Time dependence:** Often, sequential data is observed over time (e.g., time series data).
- **Contextual meaning:** The context of the sequence plays a role in understanding the data.

Real-World Examples:

- **Time Series Data:** Stock prices, weather data, temperature changes.
- **Natural Language:** Sentences or speech where the meaning of words depends on their sequence (e.g., "The cat sat on the mat").
- **Video frames:** Each frame in a video is dependent on the previous one to maintain continuity.
- **Sensor Data:** Data from IoT devices like motion sensors or temperature sensors, where each data point depends on time and prior readings.

Q2. Explain the architecture of a Recurrent Neural Network (RNN) with a neat labeled diagram.

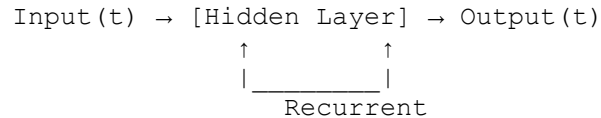
An **RNN (Recurrent Neural Network)** is designed to handle sequential data, where connections between nodes form a cycle. This allows the network to use its internal state (memory) to process sequences of inputs.

Architecture:

- **Input Layer:** Accepts sequential data, where each data point can be processed at different time steps.
- **Hidden Layer:** Has recurrent connections, allowing information to persist across time steps.
- **Output Layer:** Provides the final prediction after processing the sequence.

Diagram:

Deep Learning Assignment-043



- **Hidden State ($h(t)$):** It stores the information from previous time steps.
- The output is calculated at each time step based on the current input and the hidden state.

Q3. Discuss the structure and role of the gated memory cell in an LSTM network. How do its gates help control information flow?

LSTM (Long Short-Term Memory) networks solve the vanishing gradient problem in traditional RNNs. LSTM cells have a more complex structure than standard RNN cells due to the introduction of **gates** to control the flow of information.

Structure:

- **Forget Gate:** Decides what information from the previous cell state should be discarded.
- **Input Gate:** Controls what new information is stored in the cell state.
- **Cell State:** A memory of long-term dependencies, modified by the forget and input gates.
- **Output Gate:** Decides what part of the cell state should be output to the next layer or time step.

Role of Gates:

- **Forget Gate:** Prevents irrelevant information from accumulating in the cell state.
- **Input Gate:** Ensures that only useful information is added to the memory.
- **Output Gate:** Ensures that the relevant information is passed to the next time step.

Q4. What is the role of activation functions in RNNs and LSTMs? Discuss how they influence learning dynamics.

Activation functions introduce non-linearity into the network, allowing it to model complex patterns in data. Without them, the network would only be able to learn linear relationships.

- **In RNNs:** The activation function determines the output of each time step. Commonly used activation functions in RNNs are **tanh** and **sigmoid**. These functions help the network learn complex patterns but can cause issues like the vanishing gradient problem.
- **In LSTMs:** The use of **sigmoid** in gates (forget, input, and output) helps control the flow of information in and out of the cell state. The **tanh** activation is used to scale the cell state and output, helping manage long-term dependencies.

Influence on Learning:

- The activation functions impact how well the network can learn over long sequences and how it handles vanishing or exploding gradients.

Q5. Compare traditional RNNs and LSTMs in terms of memory handling, gradient behavior, and performance on long sequences.

- **Memory Handling:**
 - **RNNs:** Struggle to retain long-term memory due to the vanishing gradient problem.
 - **LSTMs:** Use memory cells and gates to retain and control long-term dependencies.
- **Gradient Behavior:**
 - **RNNs:** Gradients can vanish or explode during training, especially in long sequences.
 - **LSTMs:** Mitigate the vanishing gradient problem through gating mechanisms that regulate the flow of information.
- **Performance on Long Sequences:**
 - **RNNs:** Perform poorly on long sequences due to difficulty in retaining information over many time steps.
 - **LSTMs:** Perform much better on long sequences because of the cell state and gating mechanisms that allow them to capture long-term dependencies.

Q6. Differentiate between Encoder and Decoder components of a sequence model with the help of a clear labeled diagram.

The **Encoder** and **Decoder** are components of sequence-to-sequence models used in tasks like machine translation, where one sequence is transformed into another.

Encoder:

- Takes an input sequence and encodes it into a context vector (a fixed-size representation of the entire sequence).
- The encoder processes the entire sequence and compresses the information into a state vector.

Decoder:

- Takes the context vector and generates the output sequence.
- It is typically used with attention mechanisms to focus on different parts of the input sequence at each step.

Diagram:

Input Sequence (Encoder) $\rightarrow$ [Encoder RNN/LSTM] $\rightarrow$ Context Vector
Context Vector $\rightarrow$ [Decoder RNN/LSTM] $\rightarrow$ Output Sequence

Q7. Explain the Encoder-Decoder architecture used in sequence-to-sequence learning. How does it enable tasks such as translation and summarization?

The **Encoder-Decoder architecture** is used in sequence-to-sequence learning tasks, where one sequence is mapped to another. It typically consists of two parts: the encoder and the decoder, both of which can be RNNs or LSTMs.

How it works:

1. **Encoder:** Processes the input sequence and compresses it into a fixed-size vector or context vector.
2. **Decoder:** Uses the context vector to generate the output sequence, step by step.

Role in Tasks:

- **Translation:** The encoder processes a sentence in one language (e.g., English), and the decoder generates the corresponding sentence in another language (e.g., French).
- **Summarization:** The encoder processes the entire document, and the decoder generates a short summary.

Attention Mechanism: In more advanced models, attention mechanisms help the decoder focus on different parts of the input sequence at each time step, improving translation and summarization accuracy.

Part B – Numerical / Analytical

Q8. Derive the mathematical expressions for a basic RNN cell, showing how the hidden state and output are computed.

The basic RNN cell computes the hidden state at each time step based on the input at the current time step and the previous hidden state.

1. **Hidden State Computation:**
At each time step (t), the hidden state (h_t) is computed as:

$$\begin{bmatrix} h_t = \tanh(W_{\{xh\}} x_t + W_{\{hh\}} h_{\{t-1\}} + b_h) \end{bmatrix}$$

Where:

- (x_t) is the input at time step (t) .
- $(h_{\{t-1\}})$ is the hidden state at the previous time step.
- $(W_{\{xh\}})$ is the weight matrix between the input (x_t) and the hidden state.
- $(W_{\{hh\}})$ is the weight matrix between the previous hidden state $(h_{\{t-1\}})$ and the current hidden state.
- (b_h) is the bias term.

2. **Output Computation:**

The output (y_t) at each time step is computed as:

$$\begin{bmatrix} y_t = W_{\{hy\}} h_t + b_y \end{bmatrix}$$

Where:

- (y_t) is the output at time step (t) .
- $(W_{\{hy\}})$ is the weight matrix between the hidden state (h_t) and the output.
- (b_y) is the bias term for the output.

The hidden state function contains a non-linearity, typically the **tanh** function, which introduces the ability to learn complex patterns in the sequence.

Q9. Write and explain the LSTM gating equations, including forget, input, output gates, and cell state update.

The **LSTM (Long Short-Term Memory)** cell is designed to overcome the limitations of traditional RNNs, such as the vanishing gradient problem, by introducing gates to control the flow of information.

Here are the gating equations for an LSTM:

1. **Forget Gate:**

The forget gate determines how much of the previous cell state $(C_{\{t-1\}})$ should be carried forward to the next time step.

$$\begin{bmatrix} f_t = \sigma(W_f x_t + U_f h_{\{t-1\}} + b_f) \end{bmatrix}$$

Where:

- (f_t) is the forget gate's output at time step (t) .
- (σ) is the sigmoid activation function.
- (W_f) and (U_f) are the weight matrices for the input and previous hidden state.
- (b_f) is the bias term for the forget gate.

2. **Input Gate:**

The input gate controls how much of the current input (x_t) and the previous hidden state (h_{t-1}) will influence the cell state (C_t) .

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

Where:

- (i_t) is the input gate's output at time step (t) .
- (W_i) and (U_i) are the weight matrices for the input and previous hidden state.
- (b_i) is the bias term for the input gate.

3. **Candidate Cell State:**

The candidate cell state $(\tilde{C}_t)$ is a potential update to the cell state. It is created by applying the **tanh** activation function to the current input and the previous hidden state.

$$\tilde{C}_t = \tanh(W_C x_t + U_C h_{t-1} + b_C)$$

Where:

- $(\tilde{C}_t)$ is the candidate cell state at time step (t) .
- (W_C) and (U_C) are the weight matrices.
- (b_C) is the bias term.

4. **Cell State Update:**

The cell state (C_t) is updated by combining the forget gate and the input gate along with the candidate cell state.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Where:

- (C_t) is the updated cell state.
- The term $(f_t \cdot C_{t-1})$ forgets part of the previous cell state.

- The term $(i_t \cdot \tilde{C}_t)$ adds the new candidate cell state.
5. **Output Gate:**
The output gate determines what portion of the cell state should be output to the next layer and time step. It is computed using the cell state and the previous hidden state.

$$[o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)]$$

Where:

- (o_t) is the output gate's output at time step (t) .
 - (W_o) and (U_o) are the weight matrices for the input and previous hidden state.
 - (b_o) is the bias term for the output gate.
6. **Final Output:**
The final hidden state (h_t) is computed using the output gate and the updated cell state:

$$[h_t = o_t \cdot \tanh(C_t)]$$
